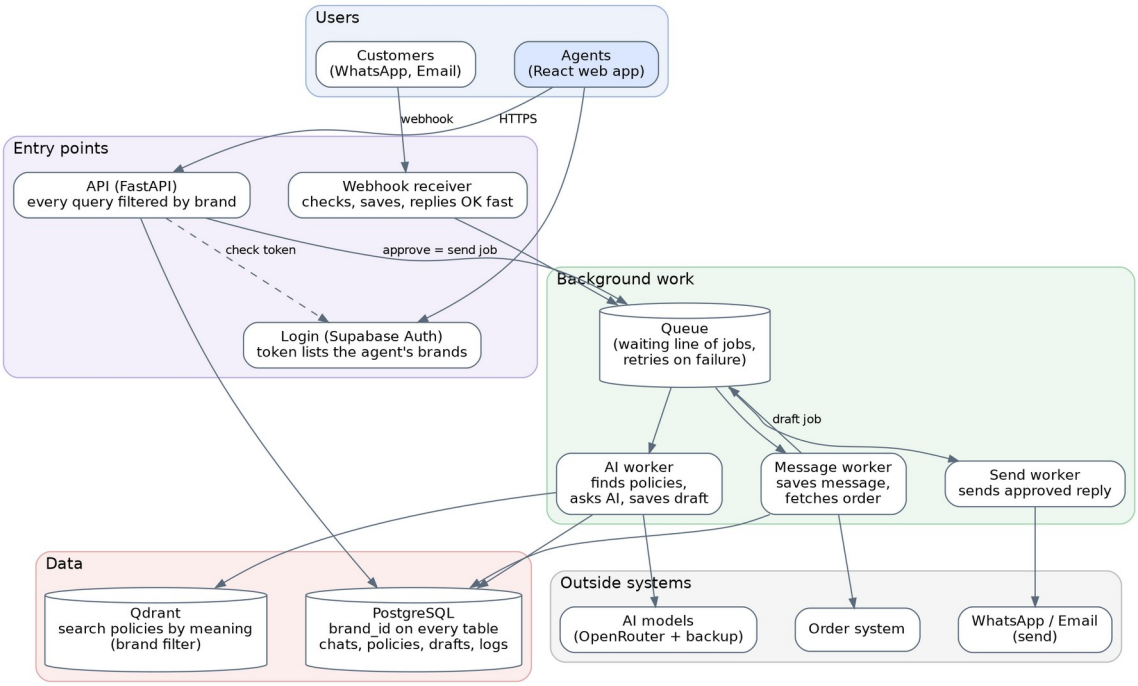
Architecture: AI-Powered CX Reply Assistant

What I built is a small version of this system. At 500 brands and 5,000 agents the flow stays the same, but the work is split into separate pieces that can each grow on their own.



1. How a message flows

1. A customer message arrives from WhatsApp or email. The sender calls our webhook URL.
2. The webhook receiver checks it, saves it, and answers OK straight away. No slow work happens here.
3. The message goes into a queue, which is a waiting line of jobs.
4. A worker saves the message and fetches the customer's order from the order system.
5. An AI worker finds that brand's policies, asks the AI for a draft reply, and saves the draft.
6. The agent sees the draft, edits it if needed, and approves it.
7. A send worker delivers the approved reply on WhatsApp or email.

2. The parts and why

Part	What I would use	Why
Frontend	React app on Vercel	Agents only click and read. All the logic stays in the backend.
Backend	FastAPI, no data kept in memory	Easy to run many copies when traffic grows.
Database	PostgreSQL (Supabase)	One reliable place for brands, chats, policies and logs.
Login	Supabase Auth	Gives each agent a token that says which brands they can open.
Integrations	Webhooks in, a send worker out	If WhatsApp or the order system is slow, the rest of the app is not blocked.
AI	One module that calls OpenRouter, plus a backup model	One place to control models, retries and cost.
Policy search	Qdrant (search by meaning), with a brand filter	Keyword search is fine for a few policies, but not for 500 brands with big knowledge bases.
Background jobs	A queue and workers	No message is lost, and busy periods just wait in line.

3. Keeping each brand's data separate

Brand A must never see Brand B. I use several layers, so one mistake does not leak data:

- **brand_id everywhere.** Every table has a brand_id column, and every query filters on it.

- **The brand is never taken from the browser.** It comes from the login token or from the conversation record.
- **Database rules as a backup.** Row Level Security hides other brands' rows even if the code forgets a filter.
- **Policy search and caches.** Policy search always adds the brand filter. Cache keys and logs also include the brand.

In my prototype every policy query filters by brand_id, and the AI only receives the policies of the conversation's brand. I checked this: Rahul's reply (HydraPure) showed only HydraPure policies.

4. Making the AI reliable

- **Right knowledge.** Only that brand's policies are sent. Related policies go together. While testing, a refund question missed the return policy that held the 48-hour damage rule, so the AI answered confidently with incomplete information. I fixed it in the search step, not in the prompt.
- **Hallucinations (the AI making things up).** The prompt says: use only these policies, and never promise a refund unless the policy clearly allows it. The reply must come back in a fixed format, and anything in the wrong format or with made-up policy ids is rejected.
- **Small, exact context.** The AI gets the brand, the order, the matching policies and the last few messages. Facts like days since delivery are calculated by our code, because AI is unreliable at date maths.
- **Confidence and fallback.** No policy found: the AI is not called, and the agent sees a warning. AI unsure: the draft is marked Needs review. A human always approves before anything is sent. If the AI fails, the agent writes the reply manually.
- **Checking quality.** Keep a list of tricky questions (outside the time limit, no policy, another brand's wording) and run them again after every prompt or model change. Also track how often agents edit the AI draft.

5. From 20 to 500 brands: what breaks first

- **1. Database connections.** My prototype opens a new database connection for every query. With many users the database runs out of connections. Fix: reuse connections (a connection pool) and add an index on brand_id.
- **2. AI limits and cost.** More brands means more AI calls at the same time, and the provider starts refusing some. Fix: limits per brand, save repeated answers, use a cheaper model for simple questions, and keep a backup model.
- **3. Policy search.** Keyword matching starts missing things when knowledge bases get big. Fix: search by meaning (Qdrant) with the brand filter.
- **4. One busy brand slowing the others.** Fix: share the queue fairly between brands.

6. What if something fails

- **A webhook arrives twice.** Each message has an ID from the sender. We store it with a unique rule, so the second copy is ignored.
- **An outside API times out.** Wait a short time and retry a few times. If it still fails, create the draft anyway and mark “order info unavailable”, so the AI does not guess.
- **An AI request fails.** Retry once, then use the backup model. If both fail, the agent sees “AI unavailable” and writes manually. The customer's message is already saved, so nothing is lost.
- **The reply is processed but not sent.** Save the reply as pending first, then send it, then mark it sent. A worker retries anything still pending. Each send has a unique ID, so a retry never sends twice. If it still fails, the agent sees “failed to send” and can resend.